

E-Commerce System Design Document

1. System Architecture

The project strictly adheres to Object-Oriented Programming (OOP) principles to ensure scalability and clean code. The architecture is divided into three primary layers:

Data Models

- Product** : Represents a physical item in the store. Encapsulates properties like `id`, `name`, `category`, and `price`.
- Coupon** : Represents a discount code. Encapsulates `code` and `discountPercentage`.
- CartItem** : Acts as a bridge between the user's cart and a `Product`. It tracks the `product` reference and the `quantity` selected.

Business Logic

- Cart** : The core logic engine. It manages a List of `CartItem` objects. It is responsible for calculating subtotals, applying percentages via a stored `Coupon`, and generating the final receipt string.

Infrastructure & Utilities

- FileHandler** : Responsible for Disk I/O. It reads from `products.csv` and `coupons.csv`, safely parses the text data into Java objects, and writes the final receipt to the `output/` directory.
- Utils** : A helper class containing the `formatCurrency` method, ensuring all prices are universally formatted to Indian Rupees (INR).
- main** : The driver class that links the Data, Logic, and Infrastructure together through an interactive command-line loop.

2. Comprehensive Pseudocode

2.1 Data Models

Product.java

```
CLASS Product
  PROPERTIES: id, name, category, price

  CONSTRUCTOR(id, name, category, price):
    ASSIGN all parameters to respective properties

  GETTERS for all properties

  FUNCTION toString():
    RETURN name + " (" + category + ") - " + Utils.formatCurrency(price)
END CLASS
```

Coupon.java

```
CLASS Coupon
  PROPERTIES: code, discountPercentage

  CONSTRUCTOR(code, discountPercentage):
    ASSIGN parameters to properties

  GETTERS for all properties
END CLASS
```

CartItem.java

```

CLASS CartItem
  PROPERTIES: product, quantity

  CONSTRUCTOR(product, quantity):
    ASSIGN parameters to properties

  GETTERS and SETTERS for properties

  FUNCTION getTotalPrice():
    RETURN product.price * quantity
END CLASS

```

2.2 Business Logic

Cart.java

```

CLASS Cart
  PROPERTIES: items (List of CartItem), appliedCoupon

  CONSTRUCTOR():
    Initialize items as empty List

  FUNCTION addItem(product, quantity):
    FOR EACH item IN items:
      IF item.product.id == product.id THEN
        item.quantity = item.quantity + quantity
      RETURN
    END IF
  END FOR
  Create new CartItem(product, quantity)
  Add new CartItem to items

  FUNCTION applyCoupon(coupon):
    SET appliedCoupon = coupon

  FUNCTION calculateSubtotal():
    subtotal = 0
    FOR EACH item IN items:
      subtotal = subtotal + item.getTotalPrice()
    RETURN subtotal

  FUNCTION calculateDiscount():
    IF appliedCoupon IS null THEN RETURN 0
    RETURN calculateSubtotal() * (appliedCoupon.discountPercentage / 100)

  FUNCTION calculateTotal():
    RETURN calculateSubtotal() - calculateDiscount()

  FUNCTION generateReceipt():
    receipt = "Receipt Header"
    FOR EACH item IN items:
      APPEND item details and formatted total price to receipt
    APPEND subtotal, discount, and final total to receipt
    RETURN receipt
END CLASS

```

2.3 Infrastructure & Utilities

FileHandler.java

```

CLASS FileHandler
  FUNCTION loadProducts(filePath):
    Create empty productsList
    TRY to open filePath
      Skip header line
      FOR EACH line IN file:
        Split line by comma
        IF data is valid THEN
          Create new Product
          Add to productsList
        END IF
      END FOR
    CATCH exceptions
      Print error
    RETURN productsList

  FUNCTION loadCoupons(filePath):
    Create empty couponsList
    TRY to open filePath
      Skip header line
      FOR EACH line IN file:
        Split line by comma
        IF data is valid THEN
          Create new Coupon
          Add to couponsList
        END IF
      END FOR
    CATCH exceptions
      Print error
    RETURN couponsList

  FUNCTION writeOutput(filePath, content):
    TRY to open filePath for writing
      Write content to file
    CATCH exceptions
      Print error
END CLASS

```

Utils.java

```

CLASS Utils
  FUNCTION formatCurrency(amount):
    Initialize NumberFormat for Indian locale ("en", "IN")
    RETURN "INR " + formatted amount
END CLASS

```

2.4 Main Application Flow

Main.java

```
CLASS Main
FUNCTION main():
    Load products using FileHandler.loadProducts()
    Load coupons using FileHandler.loadCoupons()

    Initialize Cart
    Initialize Scanner (Checks for test file first, otherwise System.in)

    WHILE true DO
        Display available products
        Prompt user for Product ID
        IF input is 'checkout' THEN BREAK

        Find product in products list
        IF product not found THEN CONTINUE

        Prompt user for quantity
        IF quantity > 0 THEN
            Cart.addItem(product, quantity)
        END IF

        Prompt user to add more products (yes/no)
        IF input is 'no' THEN BREAK
    END WHILE

    Prompt user for Coupon Code
    IF coupon code provided THEN
        Find coupon in coupons list
        IF coupon found THEN Cart.applyCoupon(coupon)
    END IF

    receipt = Cart.generateReceipt()
    Print receipt to Console
    FileHandler.writeOutput(receipt)
END CLASS
```